ECE 30862 OOP in C++
Lab III Chess – Overview and Tips

Copyright 2023-2026 30862/39595 Teaching Staff and Andrew Gan

Class Schedule (next week)

Class Schedule Week of Midterm 1	
Monday, Mar. 2	No class. Day of Exam. Office hours will be held from 4pm-6pm not 6pm-8pm. Exam: 6:30pm-7:30pm, LOCATION: FRNY G140
Tuesday, Mar. 3	N/A
Wednesday, Mar. 4	Class as usual. Office hours as usual.
Thursday, Mar. 5	Office hours as usual.
Friday, Mar. 6	Class as usual.

Scope for Each Part of the Chess Lab

Lab Components	**Content**	**Chessboard Functions**	**ChessPiece Functions**
Part I (Mar. 13)	• Chess board initialisation • Movement logic • Chess piece storage	*createChessPiece* *isValidMove*	All except for KingPiece
Part II (Mar. 27)	• Movement logistics • Threat detection	*movePiece* *isPieceUnderThreat*	All except for KingPiece
Part III (Apr. 3)	• Checking mechanism for the King		Functions belonging to KingPiece

Starter Code Constraints

- Do not change any of the public functions in the starter code.

- You can extend the header file appropriately.

- Remember to define every function so that your code at least compiles!

Config Files for Testing

- Config files are used for grading; we have provided a subset of them

- **Test Commands** use your chessboard functions to check if their output aligns with instructor's implementation
  - movePiece <**current** row> <**current** column> <**target** row> <**target** column>
    - Move piece in the chess board.
    - Should return false upon if invalid move
  - isValidScan
    - Calls isValidMove() from every position to every other position on the chess board for all pieces.
  - underThreatScan
    - Calls isPieceUnderThreat for all chess pieces.

Config Files for Testing - 2

```
0
4 4                    //  chess board of size 4x4
w r 2 3                //  place white rook at (2,3)
b b 1 1                //  place black bishop at (1,1)
~                      //  test begins
isValidScan
// below fails because it is the white pieces' turn
movePiece 1 1 0 0      // move black bishop from (1,1) to (0,0)
movePiece 2 3 4 3      // moves white rook from (2,3) to (4,3)
movePiece 1 1 3 3      // moves black bishop from (1,1) to (3,3)
// below fails due to obstruction of a piece
movePiece 4 3 2 3      // move white rook from (4,3) to (2,3)
```

Config Files for Testing - 3

```
0
4 4                    //  chess board of size 4x4
w r 2 3                //  place white rook at (2,3)
b b 1 1                //  place black bishop at (1,1)
// caution: you should overwrite existing piece
b r 1 1                //  place black rook at (1,1)
~
isValidScan
movePiece 2 3 4 3      // move white rook from (2,3) to (4,3)
// fails as it is a rook, not a bishop
movePiece 1 1 3 3      // move black rook from (1,1) to (3,3)
movePiece 1 1 3 1      // move black rook from (1,1) to (3,1)
// checks if any piece is under threat
isUnderThreatScan
```

Recreating board state from a config file

Config file content:

```
0
4 4
w r 3 2
b b 1 3
b r 1 1
w r 2 3
~
isValidScan
```

```
void test_part1_4x4_1()
{
  // Corresponding code
   Student::ChessBoard sBoard(4, 4);
   sBoard.createChessPiece(White, Rook, 3, 2);
   sBoard.createChessPiece(Black, Bishop, 1, 3);
   sBoard.createChessPiece(Black, Rook, 1, 1);
   sBoard.createChessPiece(White, Rook, 2, 3);

   // Calls isValidMove() from every position to
   // every  other position on the chess board for
   // all pieces.
   return;
}
```

Unit Test !!

- Test *individual functions* of your program in isolation

E.g., after implementing bishop, do unit test like this

```
int main(int argc, char** argv) {
        unit_isValidMove_bishop_one();
        unit_isValidMove_bishop_two();

        return EXIT_SUCCESS;
}
```

```
void unit_isValidMove_bishop_one() {
        Student::ChessBoard board(4, 4);
        board.createChessPiece(Black, Bishop, 0, 0);

        assert(board.isValidMove(0, 0, 3, 3));
        assert(!board.isValidMove(0, 0, 1, 2));
        assert(!board.isValidMove(0, 0, -1, -1));
}

void unit_isValidMove_bishop_two() {
        Student::ChessBoard board;
        board.createChessPiece(Black, Bishop, 0, 0);
        board.createChessPiece(White, Bishop, 3, 3);

        assert(board.isValidMove(0, 0, 2, 2));
        assert(board.isValidMove(0, 0, 3, 3));
        assert(!board.isValidMove(0, 0, 4, 4));
}
```

```
./chess
chess: main.cc:9: void unit_isValidMove_bishop_one(): Assertion `board.isValidMove(0, 0, 3, 3)' failed.
Aborted (core dumped)
```

Visualizing board state

- Visualizing the board can be quite helpful for debugging.
- In your own local testing, you can visualize board state using the displayBoard() function. Example: specify in main.cc:
  - std::cout << myboard.displayBoard()**.str()** << std::endl;
  - You need to implement toString() function for each piece to utilize this displayBoard() function
- When you fail a test case on Gradescope, you may see something similar to the right.
- The bottom graph shows the board state just before the failure.
- Note that for Part 1, the initial and final state would be the same as actual moves are not made.

```
Initial state:
  0 1 2 3
  - - - -
0|♟     |
1|♖ ♗   |
2|     ♗|
3|      |
 - - - -

A function call did not match expected
output at line 8 (isValidScan)
Final state:
 0123
 - - - -
0|♟     |
1|♖ ♗   |
2|     ♗|
3|      |
 - - - -
```

What if you got an error on Gradescope for provided configs?

- Verify your logic in canMoveToLocation(). Double check the rules for each piece.
- **Unit tests** for the functionality of each piece can be helpful.
- For Parts 2 and 3:
  - Draw the initial state on paper.
  - Follow the instruction in the config file,
  - Check if the final state on paper matches with the one shown on Gradescope.
  - Here, it would be advisable to test smaller boards (e.g., 4x4), or boards with fewer pieces first.
  - If useful, you can recreate the scenario in the config file for your local testing

Error information you may see on Gradescope – 1

- Suppose the code failed at a line that performs a scan

```
0
4 4
w r 2 3
b b 1 1
~
isValidScan
```

- For a subset of Gradescope test cases, we have turned on error message functionality.
- You may see an error message like this:
  - **A function call did not match expected output at line 6 (isValidScan)**
- This indicates a failure of isValidScan.
  - ‘isValidScan’ calls isValidMove for every piece to every possible position on the board
  - One of the invocations to isValidMove failed and did not match our implementation.

Error information you may see on Gradescope – 2

- "Not successful run"
  - This usually means that the test did not execute successfully.
  - A possible cause is a runtime error. Students should focus more on segmentation fault in this case.

- "ChessBoard->board appears to be out of sync at line X”
  - When trying to build up the initial board by creating new pieces and placing them on the board, the student's board did not match the instructor's board.
  - Therefore, student should focus on function related to each Pieces' constructor, and ChessBoard's createChessPiece().

Before getting started on the lab

- Read the handout carefully
- Read the comments in the header files
- Familiarise yourself with the functions available
- Understand the inheritance structure of the classes
- Start early